Manual de CI/CD com GitHub e Coolify

BiblioDist — Sistema de Gerenciamento de Biblioteca Distribuída

Disciplina: Sistemas Operacionais II
Professor: Douglas
Instituição: FEMA — Fundação Educacional do Município de Assis
Aluno: Arthur Naoto Miura
RA: 2311600029
Data: Novembro de 2026

> ***Agradecimento:*** *Este trabalho contou com o auxílio do aluno* ***Uryel****, que colaborou colocando os arquivos no diretório correto do servidor durante a etapa de deploy no Coolify.*

Sumário

1. Visão Geral do Fluxo CI/CD
2. Criação do Repositório no GitHub
3. Estrutura do Projeto no GitHub
4. Criação do Projeto no Coolify
5. Integração GitHub Actions com Coolify
6. Fluxo Completo de Deploy Automático
7. Acesso à Aplicação pela URL
8. Resultado Final
9. Guia Rápido do Administrador

---

1. Visão Geral do Fluxo CI/CD

O fluxo de **Integração Contínua / Entrega Contínua (CI/CD)** do BiblioDist funciona da seguinte forma:





Resumo: a cada `git push` para o branch `master`, o GitHub Actions chama automaticamente a API de deploy do Coolify (autenticada por token), que instrui o Coolify a fazer o rebuild e redeploy de todos os containers da aplicação.

---

2. Criação do Repositório no GitHub

Passo 1 — Criar o repositório

Acesse `github.com`, clique em **New repository** e configure:

Campo	Valor
Repository name	`bibliodist`
Visibility	Public
Initialize with README	Não (já temos o código local)

Clique em **Create repository**.

Passo 2 — Conectar o repositório local

No terminal, dentro da pasta do projeto:

```
git init
git add .
git commit -m "feat: commit inicial"
git branch -M master
git remote add origin https://github.com/R2nin/bibliodist.git
git push -u origin master
```

Passo 3 — Verificar o repositório

Após o push, o repositório estará acessível em:
`https://github.com/R2nin/bibliodist`

O branch principal é o `master`. Todo push para esse branch aciona o pipeline de deploy.

---

3. Estrutura do Projeto no GitHub

O repositório é um **monorepo** com um serviço por pasta. Cada serviço tem seu próprio `Dockerfile`:

```
bibliodist/
├── .github/
│   └── workflows/
│       └── deploy.yml          ← pipeline CI/CD
│
├── api-web/                    ← Django + Gunicorn
│   ├── Dockerfile
│   ├── requirements.txt
│   └── ...
│
├── grpc-service/               ← servidor gRPC
│   ├── Dockerfile
│   ├── requirements.txt
│   └── ...
│
├── worker/                     ← consumidor RabbitMQ
│   ├── Dockerfile
│   ├── requirements.txt
│   └── ...
│
├── proto/                      ← contrato gRPC (.proto)
│
├── docker-compose.yml          ← uso local/dev
├── docker-compose.coolify.yml  ← uso em produção (Coolify)
└── README.md
```

O arquivo `docker-compose.coolify.yml` é o que o Coolify usa. Nele, o `context` de cada serviço aponta diretamente para o repositório no GitHub:

```
web:
  build:
    context: https://github.com/R2nin/bibliodist.git#master
    dockerfile: api-web/Dockerfile
```

Isso significa que, ao fazer deploy, o Coolify clona o branch `master` diretamente do GitHub e compila as imagens no servidor.

---

4. Criação do Projeto no Coolify

O que é o Coolify?

Coolify é uma plataforma PaaS (Platform as a Service) *self-hosted*, similar ao Heroku ou Railway, mas instalada em servidor próprio. Permite fazer deploy de aplicações via Docker Compose com painel gráfico, certificados SSL automáticos e domínios personalizados.

A instância utilizada está instalada no servidor do professor em `computacao.fema.edu.br`.

Passo 1 — Acessar o painel Coolify

Acesse o painel em `http://computacao.fema.edu.br:8000` (ou porta configurada) e faça login com as credenciais fornecidas pelo administrador.

Passo 2 — Criar um novo projeto

No menu lateral, clique em **Projects** → **+ New Project**.

Preencha:

- **Name:** `BiblioDist`
- **Description:** Sistema de Gerenciamento de Biblioteca Distribuída

Clique em **Create Project**.

Passo 3 — Criar um novo ambiente

Dentro do projeto, clique em **+ New Environment**:

- **Name:** `production`

Passo 4 — Adicionar um novo recurso (Docker Compose)

Dentro do ambiente `production`, clique em **+ New Resource** → **Docker Compose**.

Selecione a origem:

- **Raw Docker Compose** (colar o conteúdo diretamente)

Passo 5 — Colar o `docker-compose.coolify.yml`

Cole o conteúdo do arquivo `docker-compose.coolify.yml` do repositório. Esse arquivo define os 6 serviços:

```
services:
  db:        # PostgreSQL 16
  rabbitmq:  # RabbitMQ 3.13
  migrate:   # Init container — roda migrate e sai
  web:       # Django + Gunicorn (porta 8000)
  grpc:      # Servidor gRPC (porta 50051)
  worker:    # Consumidor RabbitMQ
```

Cada serviço com `build.context` apontando para o GitHub é compilado pelo Coolify no momento do deploy.

Passo 6 — Configurar variáveis de ambiente

No painel do serviço `web`, adicione as variáveis de ambiente:

Variável	Valor
`DATABASE_URL`	`postgresql://bibliodist:bibliodist@db:5432/bibliodist`
`RABBITMQ_URL`	`amqp://guest:guest@rabbitmq:5672/`
`GRPC_HOST`	`grpc`
`GRPC_PORT`	`50051`
`DEBUG`	`False`
`ALLOWED_HOSTS`	`bibliodist.187.33.201.107.sslip.io,localhost`

Variável	Valor
`CSRF_TRUSTED_ORIGINS`	`http://bibliodist.187.33.201.107.sslip.io` (derivado automaticamente de `ALLOWED_HOSTS`)

Passo 7 — Configurar o domínio

No serviço `web`, em **Domains**, adicione:

```
http://bibliodist.187.33.201.107.sslip.io
```

O domínio `sslip.io` é um serviço DNS público que resolve qualquer subdomínio no formato `IP.sslip.io` automaticamente para o IP correspondente. Isso elimina a necessidade de configurar registros DNS manualmente.

> ***Situação atual do HTTPS:*** *o proxy do servidor responde na porta 443 com o certificado de `fema.edu.br`, que não é válido para o subdomínio sslip.io. Por isso o acesso é feito por **HTTP**. Para habilitar HTTPS basta cadastrar o domínio com `https://` em **Domains** e deixar o Coolify emitir o certificado via **Let's Encrypt** — fica como melhoria futura.*

Passo 8 — Obter a URL de deploy e o token de API

A integração precisa de **duas informações** do Coolify:

a) Deploy Webhook URL — na barra lateral do recurso, menu **Webhooks**. Formato:

```
https://<servidor-coolify>/api/v1/deploy?uuid=XXXXXXXX&force=false
```

b) API Token — no Coolify v4 a URL de deploy é um endpoint da API e **exige autenticação**. Para criar o token:

1. No menu principal do Coolify (seta `<` ao lado de *Dashboard*), abra **Keys & Tokens** → **API Tokens**.
2. Dê um nome (ex.: `github-actions`) e marque apenas a permissão **deploy**.
3. Clique em criar e **copie o token imediatamente** — ele é exibido uma única vez.

As duas informações serão cadastradas como *secrets* no GitHub no próximo capítulo.

Passo 9 — Primeiro deploy manual

Clique em **Deploy** para fazer o primeiro deploy. O Coolify vai:

1. Clonar o repositório do GitHub
2. Compilar as imagens Docker de cada serviço
3. Inicializar os containers na ordem correta (healthchecks garantem isso)
4. Rotear o tráfego via Traefik (proxy reverso interno)

Aguarde todos os containers ficarem com status **Running** (o `migrate` fica **Exited 0** — normal, é um init container).

---

5. Integração GitHub Actions com Coolify

Como funciona

O **GitHub Actions** é a plataforma de CI/CD nativa do GitHub. Um arquivo YAML em `.github/workflows/` define quando e como o pipeline é executado.

A integração com o Coolify funciona por **webhook autenticado**: o GitHub Actions envia uma requisição HTTP `POST` para a API de deploy do Coolify, com o cabeçalho `Authorization: Bearer <token>`. O Coolify valida o token e inicia o rebuild e redeploy.

Passo 1 — Cadastrar os Secrets no GitHub

No repositório GitHub, acesse: **Settings** → **Secrets and variables** → **Actions** → **New repository secret**

> *Atenção: não confundir com **Deploy keys** (chaves SSH), que fica no mesmo menu e não é usado aqui.*

Cadastre **dois** secrets:

Name	Secret
`COOLIFY_WEBHOOK_URL`	URL de deploy copiada do Coolify (Passo 8a)
`COOLIFY_TOKEN`	Token de API criado no Coolify (Passo 8b)

> *O Secret fica criptografado e nunca aparece nos logs do Actions — apenas o próprio workflow pode utilizá-lo.*

Passo 2 — O arquivo de workflow

O arquivo `.github/workflows/deploy.yml` já está no repositório:

```yaml
name: Deploy para Coolify

on:
  push:
    branches:
      - master

jobs:
  deploy:
    name: Acionar deploy no Coolify
    runs-on: ubuntu-latest

    steps:
      - name: Notificar Coolify via webhook
        run: |
          if [ -z "$COOLIFY_WEBHOOK_URL" ] || [ -z "$COOLIFY_TOKEN" ]; then
            echo "::error::Configure os secrets COOLIFY_WEBHOOK_URL e COOLIFY_TOKEN (Setting
            exit 1
          fi
          # Mostra só o caminho da URL (sem domínio nem query/token) para diagnóstico
          caminho="${COOLIFY_WEBHOOK_URL%%\?*}"
          echo "Endpoint: /${caminho#*://*/}"

          chamar() {
            curl --silent --show-error --output resposta.txt --write-out '%{http_code}' \
              --request "$1" \
              --header "Authorization: Bearer $COOLIFY_TOKEN" \
              "$COOLIFY_WEBHOOK_URL"
          }

          status=$(chamar GET)
          if [ "$status" = "405" ]; then
            echo "GET retornou 405, tentando POST..."
            status=$(chamar POST)
          fi
          echo "HTTP $status"
          head -c 500 resposta.txt; echo
          case "$status" in 2*) ;; *) exit 1 ;; esac
        env:
          COOLIFY_WEBHOOK_URL: ${{ secrets.COOLIFY_WEBHOOK_URL }}
          COOLIFY_TOKEN: ${{ secrets.COOLIFY_TOKEN }}
```

Explicação:

Elemento	Significado
`on: push: branches: master`	Aciona o pipeline a cada push no branch `master`

Elemento	Significado
`runs-on: ubuntu-latest`	Executa em um runner Linux gratuito do GitHub
Verificação inicial (`-z`)	Falha com mensagem clara se algum secret não foi cadastrado
`Endpoint: ...`	Mostra no log só o caminho da URL — nunca o token
`Authorization: Bearer`	Autentica a chamada na API do Coolify com o token
`GET` → `POST`	Tenta `GET` ; se o Coolify responder 405, repete com `POST` (o Coolify v4.7 aceita apenas `POST`)
`case "$status" in 2*`	O job só fica verde se a resposta for HTTP 2xx

Passo 3 — Verificar a execução no GitHub

Após qualquer `git push` para `master` , acesse:
GitHub → **repositório** → aba **Actions**

Você verá o workflow em execução (ícone amarelo) ou concluído (ícone verde ✓).

Clique no workflow para ver os logs de cada step. O step **Notificar Coolify via webhook** deve exibir algo como:

```
Endpoint: /api/v1/deploy
GET retornou 405, tentando POST...
HTTP 200
{"deployments":[{"message":"Service App: Arthur started. It could take a while, be patient."
```

Passo 4 — Problemas encontrados e soluções

Durante a configuração real do pipeline os seguintes erros apareceram, nesta ordem (o GitHub envia um e-mail "*Run failed*" a cada falha):

Erro no log do Actions	Causa	Solução
`curl: option : blank argument`	Secret `COOLIFY_WEBHOOK_URL` não cadastrado (chega vazio)	Cadastrar o secret (Passo 1)
`HTTP 401` (Unauthorized)	A API de deploy do Coolify exige token	Criar o API Token (Passo 8b) e cadastrar `COOLIFY_TOKEN`
`HTTP 405` (Method Not Allowed)	O Coolify v4.7 não aceita `GET` nesse endpoint	Workflow refaz a chamada com `POST`
`HTTP 200`	—	Deploy iniciado com sucesso ✓

Para repetir um workflow que falhou sem fazer novo commit: aba **Actions** → abrir a execução → **Re-run jobs** (ou `gh run rerun <id>` no terminal).

---

6. Fluxo Completo de Deploy Automático

Após toda a configuração, o fluxo se torna completamente automático:

Ciclo de desenvolvimento

```
1. Desenvolvedor edita o código localmente
         |
         ▼
2. git add . && git commit -m "feat: nova funcionalidade"
         |
         ▼
3. git push origin master
         |
         ▼
4. GitHub detecta o push → dispara o workflow deploy.yml
         |
         ▼
5. GitHub Actions executa: curl POST COOLIFY_WEBHOOK_URL
   (cabeçalho Authorization: Bearer COOLIFY_TOKEN)
         |
         ▼
6. Coolify recebe o webhook → inicia rebuild
   - Clona o repositório (branch master)
   - Compila imagens: web, grpc, worker
   - Para os containers antigos
   - Sobe os novos containers
   - Aplica migrations (init container migrate)
         |
         ▼
7. Nova versão disponível em ~2-3 minutos
   http://bibliodist.187.33.201.107.sslip.io
```

Exemplo real de deploy

Commit `9e09c10` — *ci: tentar POST no deploy do Coolify quando GET retorna 405* (14/11/2026):

1. `git push origin master`
2. GitHub Actions iniciou o workflow em poucos segundos
3. Chamada à API do Coolify: `HTTP 200` — "*Service App: Arthur started*"
4. Coolify recompilou as imagens e recriou os containers
5. Aplicação permaneceu respondendo durante o redeploy (monitorada a cada 30 s)

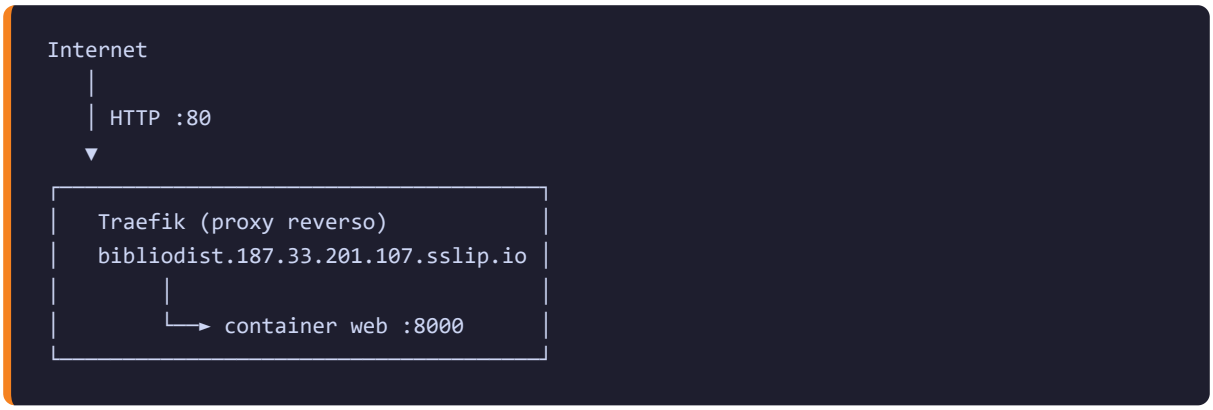
---

7. Acesso à Aplicação pela URL

Como o roteamento funciona no Coolify

O Coolify usa o **Traefik** como proxy reverso. O Traefik:

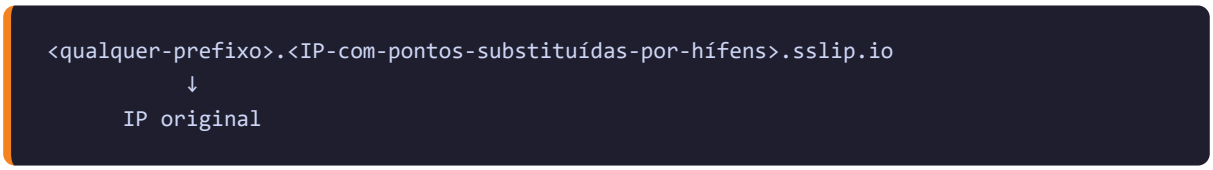
1. Escuta nas portas 80 (HTTP) e 443 (HTTPS) — o BiblioDist usa hoje a porta 80
2. Lê os labels Docker de cada container para saber qual domínio rotear
3. Pode obter e renovar certificados SSL via Let's Encrypt (ver observação sobre HTTPS no Passo 7)



O serviço `web` **não** expõe a porta 8000 diretamente na internet — apenas o Traefik tem acesso a ela internamente na rede Docker.

O domínio sslip.io

`sslip.io` é um serviço de DNS público que resolve automaticamente subdomínios no formato:



Exemplo:



Isso permite apontar um domínio para qualquer IP sem precisar configurar registros DNS — ideal para ambientes acadêmicos.

URL amigável com encurtador de links

A URL técnica gerada pelo sslip.io (`bibliodist.187.33.201.107.sslip.io`) é longa e difícil de memorizar. Para facilitar o acesso e a divulgação, foi utilizado o serviço de encurtamento de links **TinyURL**, criando uma URL amigável:

	URL
Acesso rápido (TinyURL)	`https://tinyurl.com/femabibv2`
URL técnica completa	`http://bibliodist.187.33.201.107.sslip.io`

O TinyURL redireciona automaticamente (HTTP 301) para `http://bibliodist.187.33.201.107.sslip.io/login/`, sem qualquer configuração adicional no servidor.

URLs disponíveis em produção

Serviço	URL
Aplicação principal	`https://tinyurl.com/femabibv2`
Login	`http://bibliodist.187.33.201.107.sslip.io/login/`
Painel Admin	`http://bibliodist.187.33.201.107.sslip.io/admin/`
Revisão PRISMA	`http://bibliodist.187.33.201.107.sslip.io/prisma/`
Lista de Livros	`http://bibliodist.187.33.201.107.sslip.io/livros/`

---

8. Resultado Final

Status dos containers em produção

Container	Status	Função
`db`	Running (healthy)	PostgreSQL 16 — banco de dados
`rabbitmq`	Running (healthy)	RabbitMQ 3.13 — fila de mensagens
`migrate`	Exited (0)	Init container — aplica migrations
`web`	Running	Django + Gunicorn — interface web
`grpc`	Running	Servidor gRPC — verificação de disponibilidade
`worker`	Running	Consumidor RabbitMQ — notificações assíncronas

Resumo da pipeline CI/CD

Etapa	Ferramenta	Trigger
Versionamento do código	Git + GitHub	Manual (git push)
Execução do pipeline	GitHub Actions	Automático (push no master)
Notificação de deploy	curl POST + token de API	Automático (Actions)
Build das imagens	Docker (no servidor)	Automático (Coolify)
Deploy dos containers	Docker Compose	Automático (Coolify)
Roteamento (proxy reverso)	Traefik	Automático (Coolify)

Repositório

`https://github.com/R2nin/bibliodist`

Todos os arquivos de configuração do CI/CD estão versionados no repositório:

- `.github/workflows/deploy.yml` — workflow do GitHub Actions
- `docker-compose.coolify.yml` — definição dos serviços para produção

---

9. Guia Rápido do Administrador

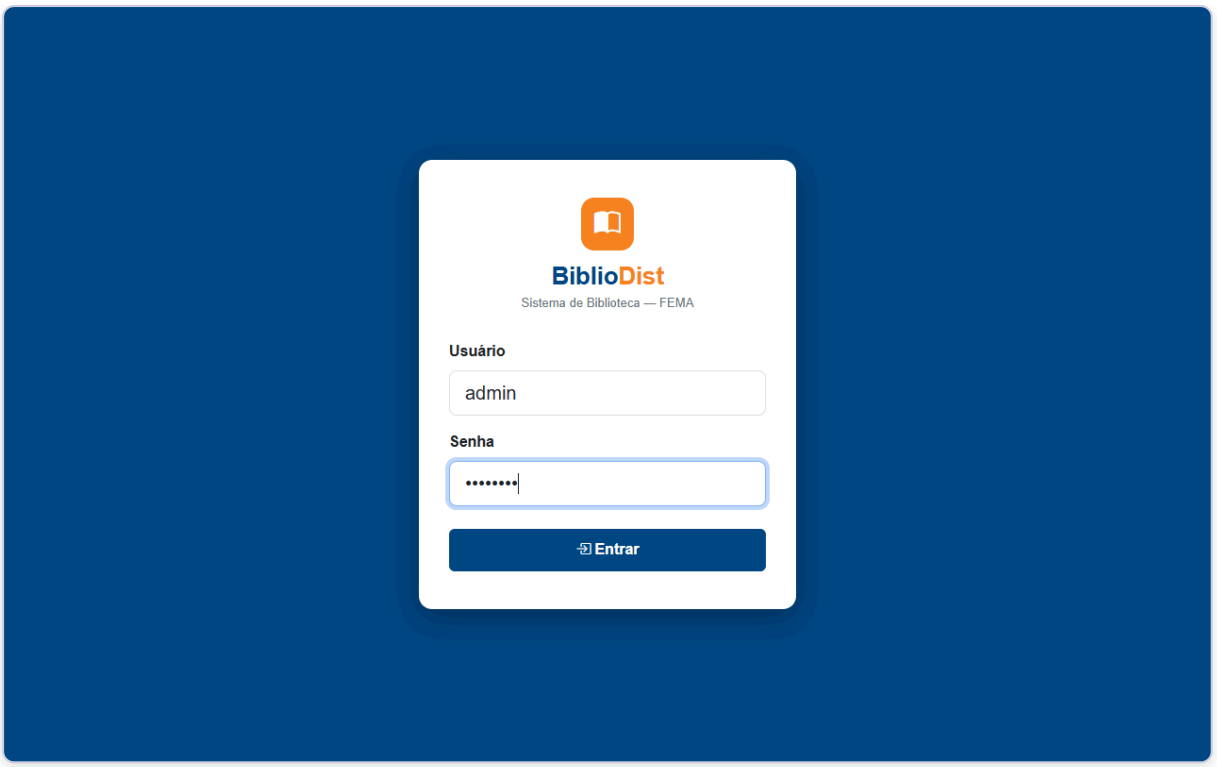
Este capítulo mostra o uso do BiblioDist com o perfil de **administrador**. As telas abaixo foram capturadas com o acervo real da biblioteca da FEMA carregado (**12.020 títulos / 36.399 exemplares**).

Perfis de acesso

Perfil	Pode fazer
Superadmin	Tudo, inclusive criar usuários e redefinir senhas (menu **Usuários**)
Administrador	Cadastrar e editar livros, autores, categorias e leitores; registrar empréstimos e devoluções; ver logs
Usuário normal	Apenas consultar o acervo (busca e detalhes dos livros)

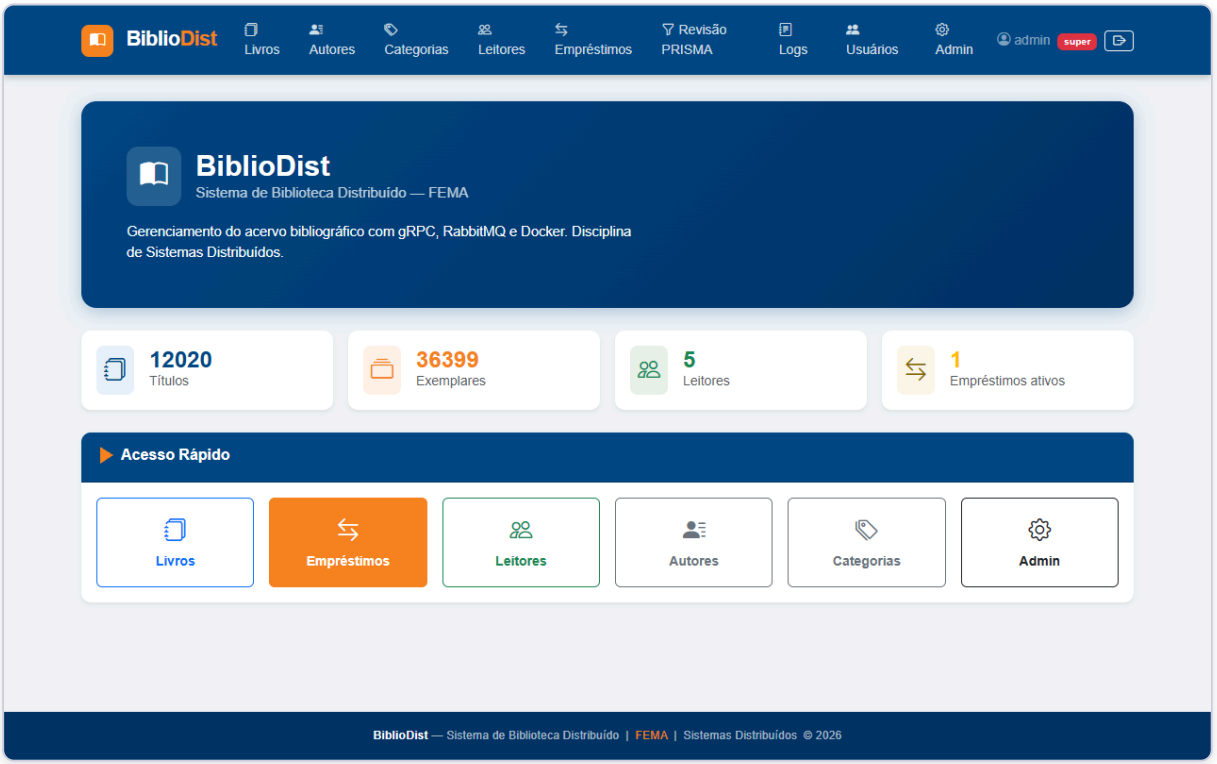
9.1 Entrar no sistema

Acesse `https://tinyurl.com/femabibv2` (ou a URL técnica) e informe usuário e senha.



9.2 Painel inicial

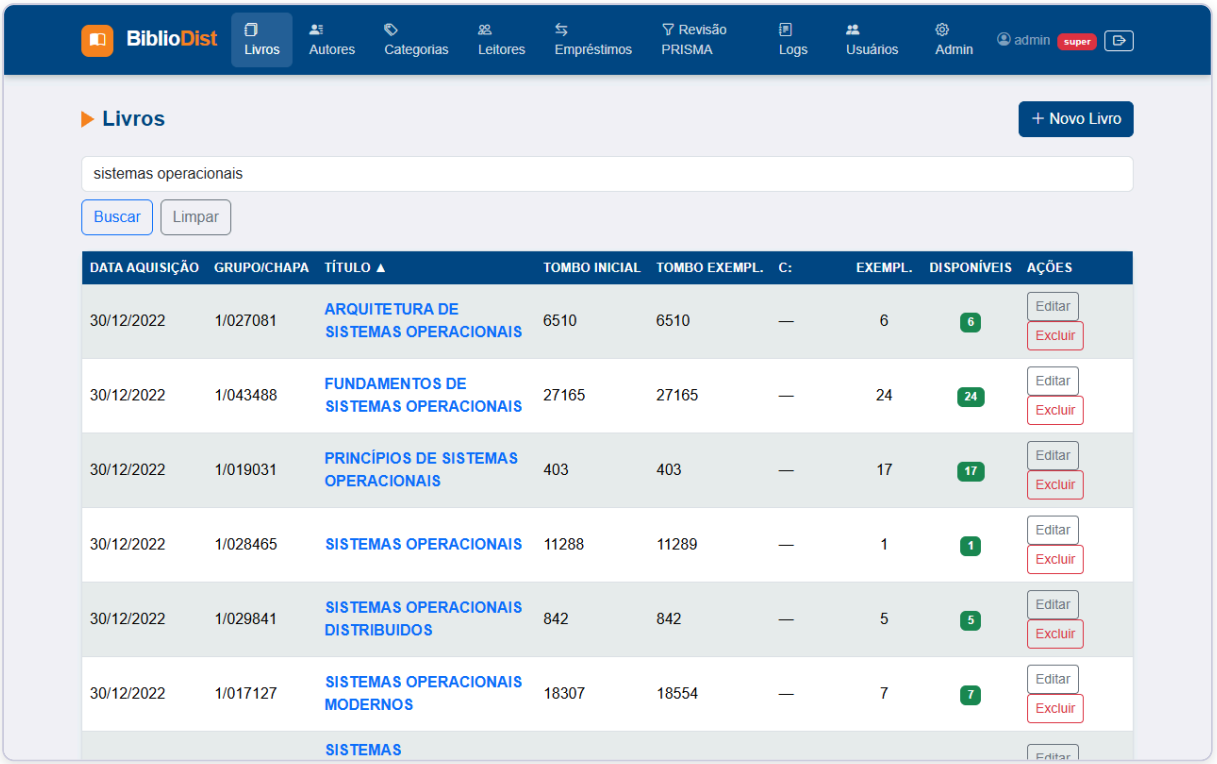
Após o login, o painel mostra os totais do acervo (títulos, exemplares, leitores e empréstimos ativos) e atalhos para as funções principais. O menu superior dá acesso a todas as telas.



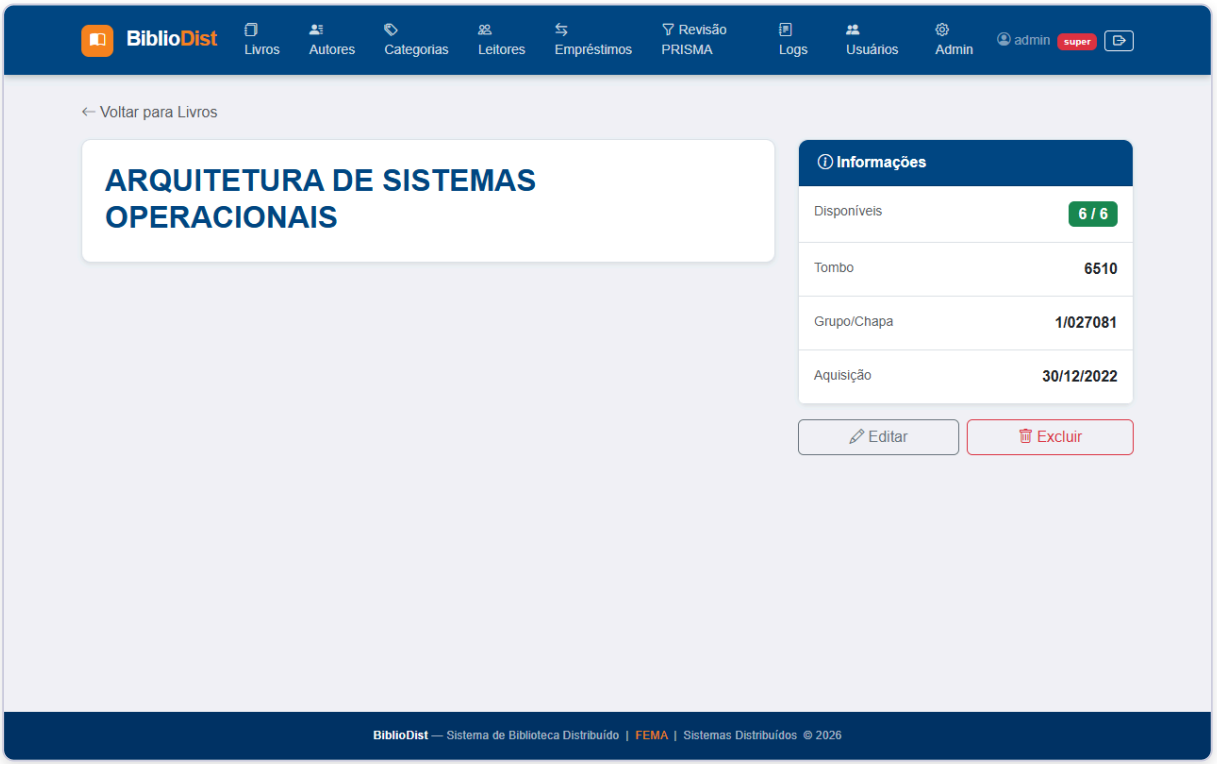
9.3 Consultar o acervo

Em **Livros**, digite parte do título, da chapa ou do tombo e clique em **Buscar**. As colunas podem ser ordenadas clicando no cabeçalho; a coluna **Disponíveis** indica quantos exemplares estão livres para

empréstimo.

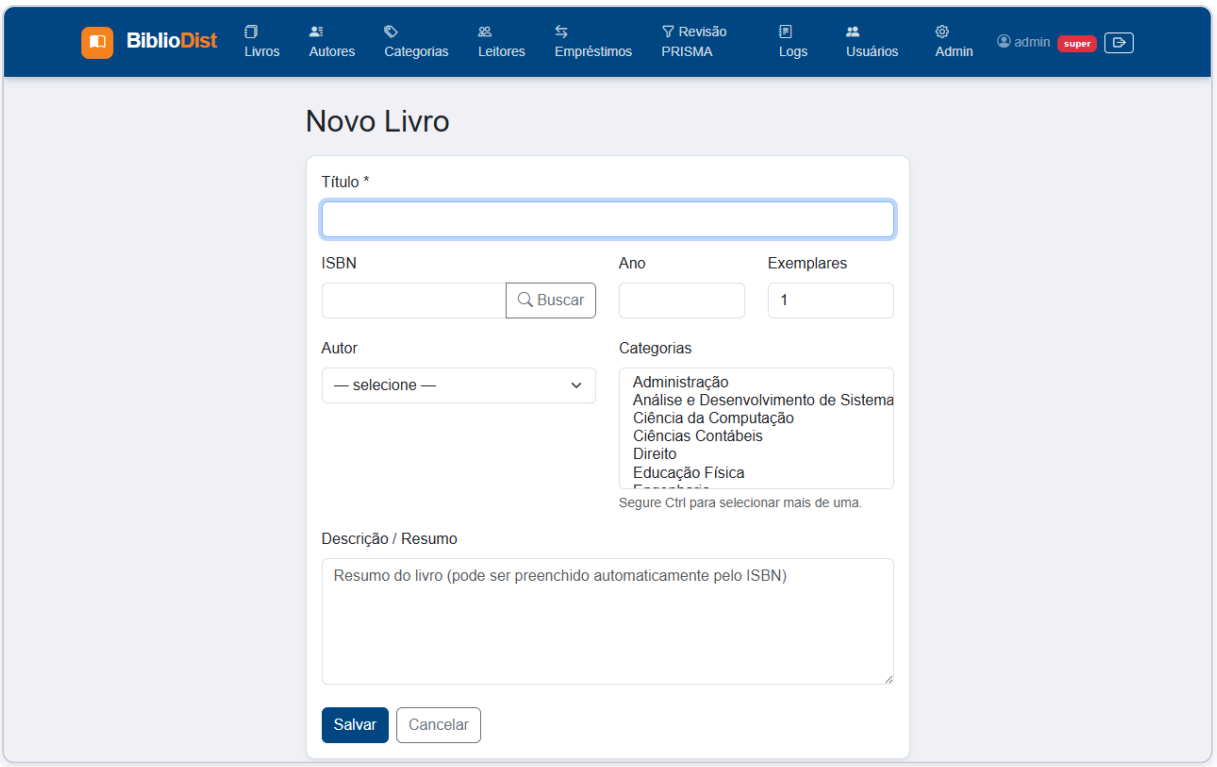


Clicando no título, abre-se a ficha do livro com tombo, grupo/chapa, data de aquisição e disponibilidade.



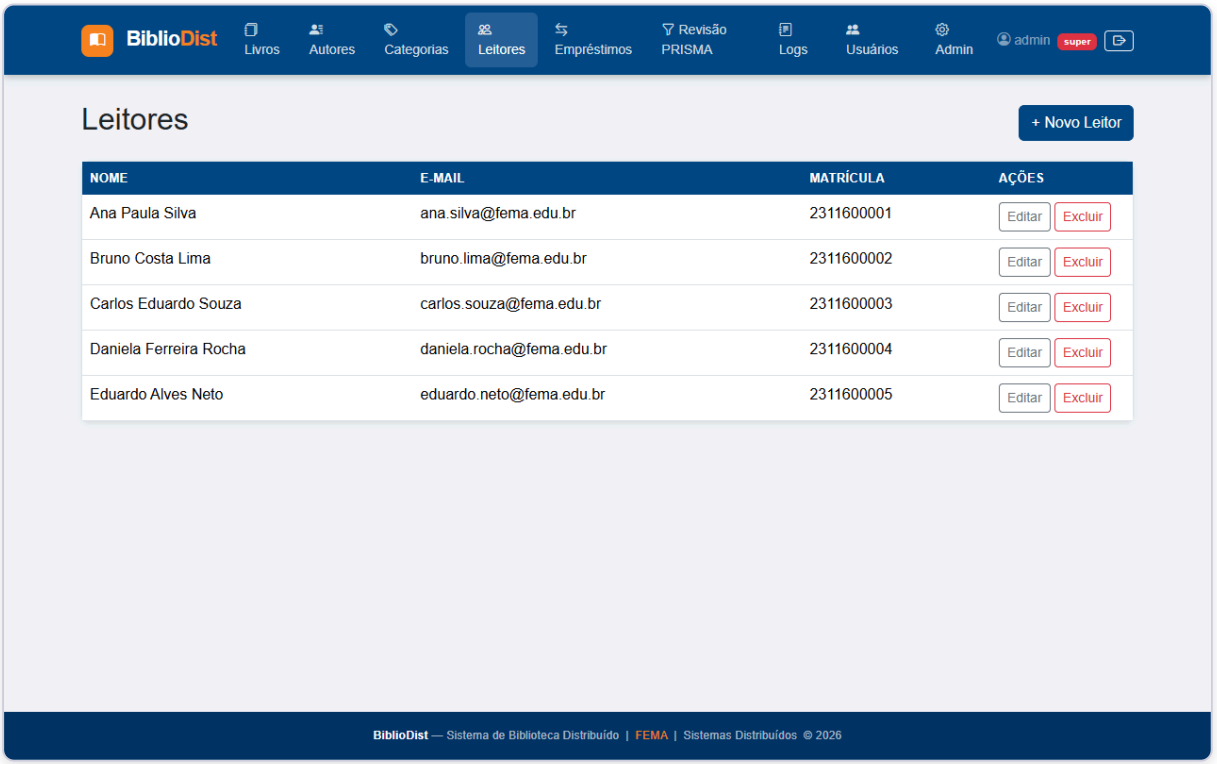
9.4 Cadastrar um livro

Em **Livros → + Novo Livro**, preencha o título e os demais dados. Informando o **ISBN** e clicando em **Buscar**, o sistema consulta o Google Books / Open Library e preenche título, ano e resumo automaticamente.



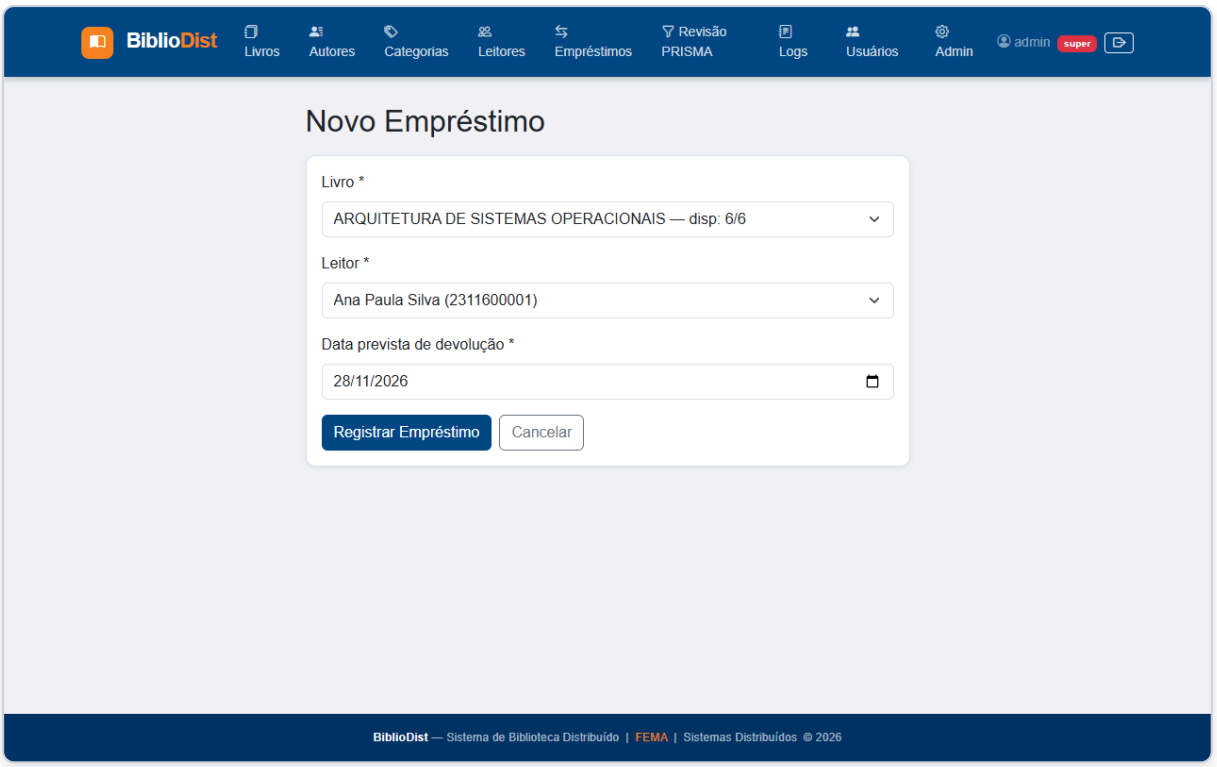
9.5 Cadastrar leitores

Em **Leitores → + Novo Leitor**, cadastre nome, e-mail e matrícula. Só leitores cadastrados podem receber empréstimos.

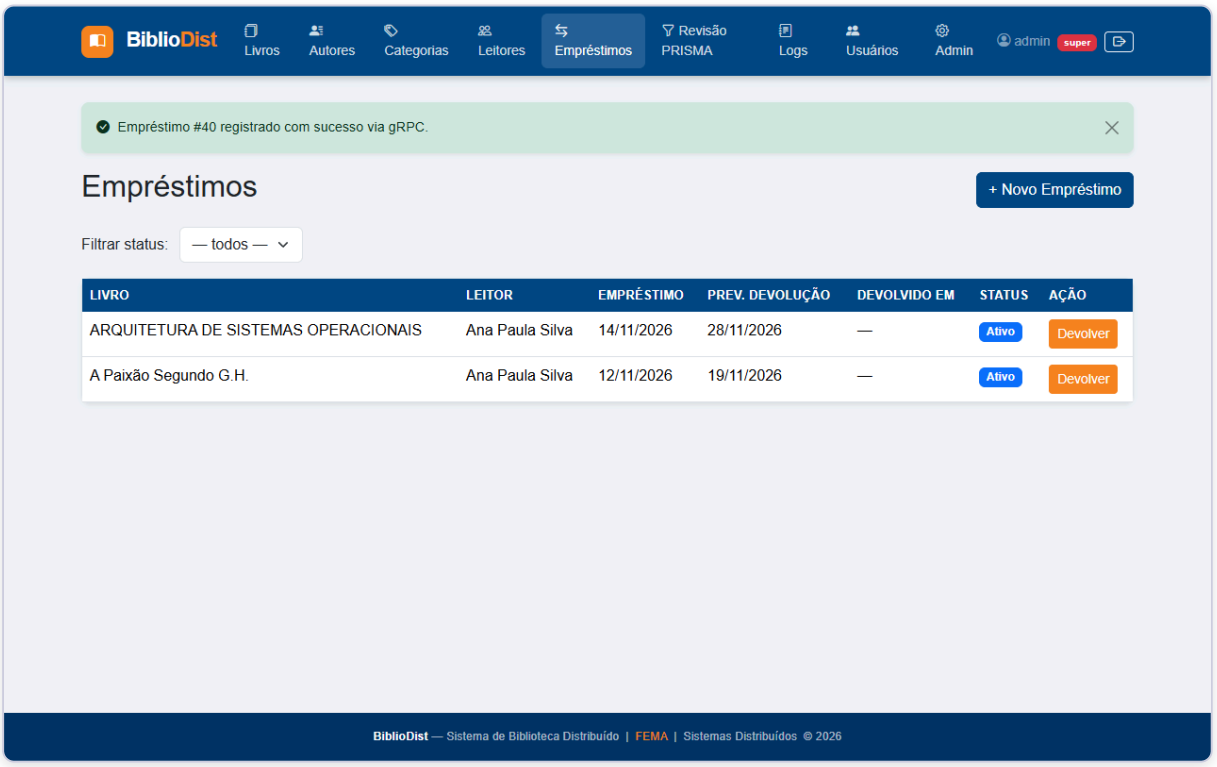


9.6 Registrar um empréstimo

Em **Empréstimos → + Novo Empréstimo**, escolha o livro (o seletor mostra a disponibilidade), o leitor e a data prevista de devolução, e clique em **Registrar Empréstimo**.

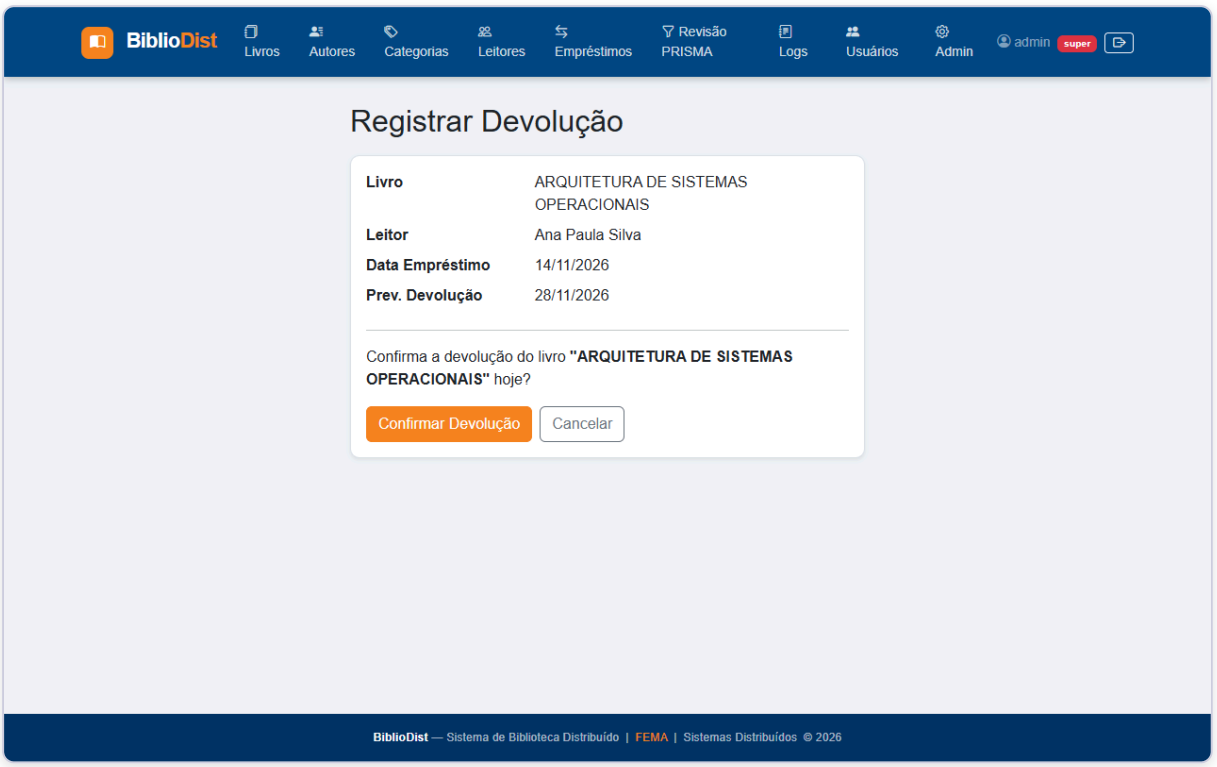


Por trás dessa tela atuam os serviços distribuídos: o Django consulta a disponibilidade e registra o empréstimo no **servidor gRPC**, que publica uma mensagem no **RabbitMQ**; o **worker** consome a mensagem e gera a notificação ao leitor. A mensagem "*registrado com sucesso via gRPC*" confirma o caminho completo. Se o gRPC estiver fora do ar, o empréstimo é gravado em **modo local** e o fato fica registrado nos logs.



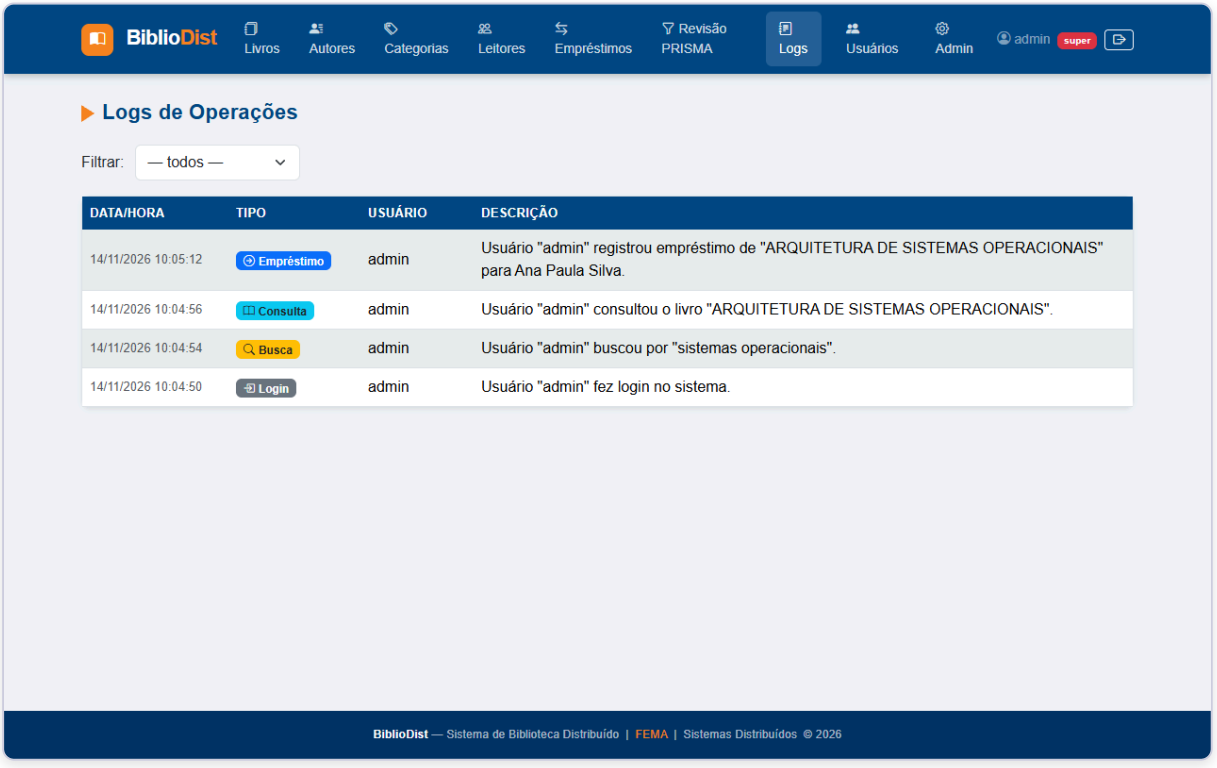
9.7 Registrar a devolução

Na lista de empréstimos, clique em **Devolver** na linha do empréstimo ativo e confirme. O exemplar volta a ficar disponível e o worker gera a notificação de devolução.



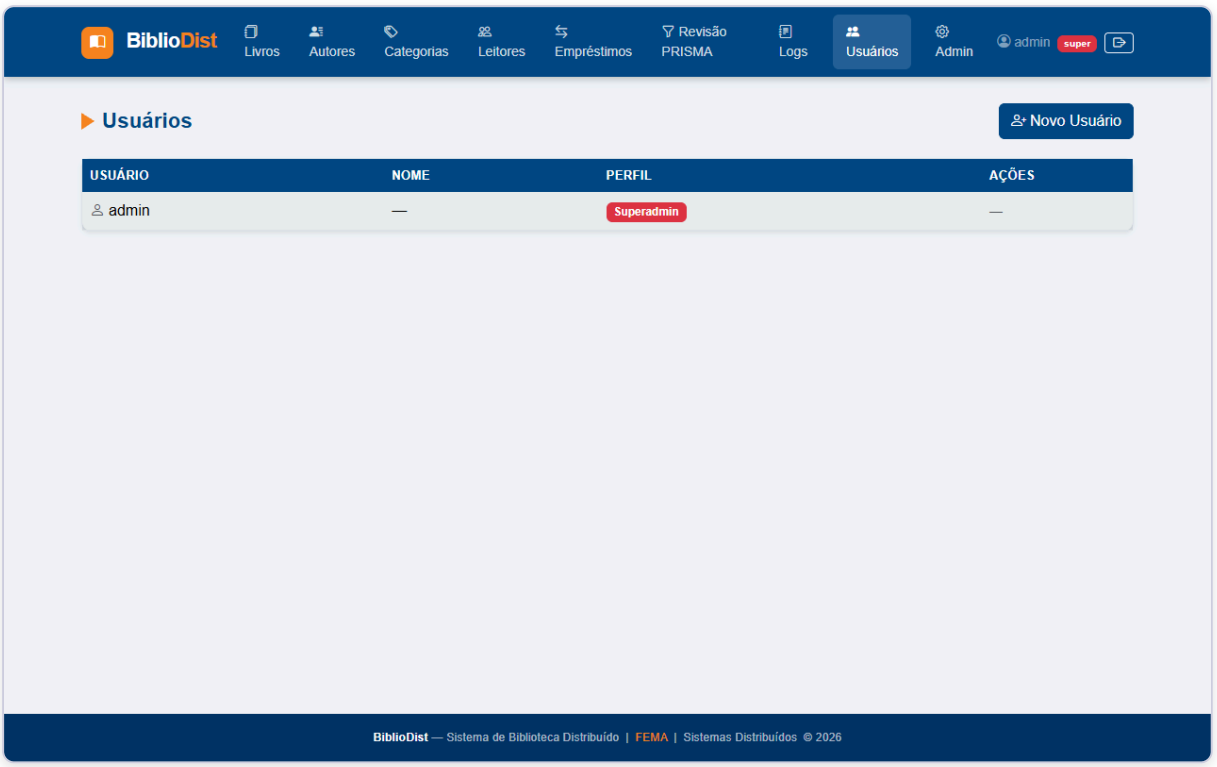
9.8 Acompanhar os logs

Em **Logs**, o administrador vê o histórico de operações — logins, buscas, consultas, empréstimos e devoluções — com data/hora e usuário. O filtro no topo permite ver apenas um tipo de operação.



9.9 Gerenciar usuários (Superadmin)

Em **Usuários → Novo Usuário**, o superadmin cria contas escolhendo o perfil (administrador ou normal) e pode redefinir senhas. Para tarefas avançadas, o menu **Admin** abre o painel administrativo do Django.



> ***Observação:*** *Este documento reflete o estado do projeto em 14 de novembro de 2026.*

Manual elaborado como parte da avaliação da disciplina de Sistemas Operacionais II — FEMA, Novembro de 2026.